

BarkahFlow — System Architecture & Implementation

BarkahFlow — System Architecture & Implementation Blueprint

Target Audience: Core Engineering Team, Cross-Platform Developers, and Technical Interns

System Class: Standalone, Offline-First cross-platform Micro-ERP / Point of Sale (POS)

1. Executive Summary & Project Intent

BarkahFlow is an Arabic-first Financial Operating System and Point of Sale (POS) designed explicitly for retail merchants, small product-based businesses, and traditional storefronts.

Unlike conventional software architectures that treat the internet as the primary source of truth, BarkahFlow inverts this relationship: the local installation is the absolute source of truth. Cloud infrastructure is strictly utilized as a gateway for user authentication and recurring subscription enforcement.

To eliminate data sync overhead, race conditions, and merge-conflict complexities, this release targets a single-device standalone environment. Each terminal operates its own secure, isolated database file. Data modification streams never sync between multiple customer devices, ensuring high data execution rates, zero network latency, and guaranteed atomic stability during high-volume operations.

2. Universal Client Bridge Architecture

To maintain code efficiency and allow a lean team to deliver both a Desktop and Mobile application, BarkahFlow leverages a Single Frontend, Dual-Native-Bridge design pattern.

2.1 The Tech Stack Matrix

- Core UI / Framework Layer: Next.js (React) styled with Tailwind CSS and Radix-based headless components (Shadcn UI). It includes native, system-wide Left-to-Right (LTR) and Right-to-Left (RTL) localization to support fluent Arabic bookkeeping.
- Desktop Target Environment: Tauri Framework (Rust backend container using the system's native WebView2/WebKit rendering layer).
- Mobile Target Environment: Capacitor runtime (wrapping the client bundle into optimized native Android and iOS application shells).

2.2 Compilation Strategy: Static SPA Delivery

Because local devices execute SQLite directly through native operating system bindings, standard Next.js Server-Side Rendering (SSR) and React Server Components (RSC) cannot run locally.

- The application is compiled entirely as a Static Single Page Application (SPA) utilizing Next.js's static export capabilities (`output: 'export'`).
- This build processes all application paths, assets, and UI components into a collection of optimized, client-side HTML, CSS, and JavaScript files housed inside a standalone output folder (`/out`).
- The Tauri configuration and the Capacitor configuration both point directly to this exact same `/out` folder. The respective shell packages bundle these static assets into their native execution contexts.

2.3 The Architectural Bridge Layer

To ensure developers write frontend business components only once, the app utilizes an abstract Database Driver Interface.

Instead of components querying a database engine directly, they communicate with a unified wrapper interface. At application boot, the factory layer detects whether the current environment runtime is Tauri (Desktop) or Capacitor (Mobile). It automatically links the appropriate native hardware plugin underneath:

1. Tauri Container: Mounts `tauri-plugin-sql`, executing raw queries through a native Rust driver directly to `libsqlite3`.
2. Capacitor Container: Mounts `@capacitor-community/sqlite`, executing raw queries via native Java/Kotlin (Android) or Objective-C/Swift (iOS) persistence pipelines.

3. Core Functional System Use Cases

Interns and developers must implement features according to these core structural system boundaries:

Use Case 1: Active Subscription Gatekeeping

- Trigger: Application Initialization / Cold Start.
- Process Flow:
 - a. The application boots into a localized loading screen and establishes an online connection to the cloud backend (Supabase).
 - b. The merchant authenticates securely using Supabase Google OAuth.
 - c. Once authenticated, an immediate real-time network request queries the cloud database (`subscriptions` table) to verify the account's validity.
 - d. If the subscription status reads `active` and the expiration timestamp is valid, the online gateway releases the interface and loads the offline database dashboard.
 - e. If the client device is completely offline but possesses a previously saved, unexpired local authentication token session, access is granted to ensure the merchant is never locked out during an internet outage. If the subscription is flagged as unpaid, expired, or canceled during an online check, the UI is immediately intercepted by a full-screen lockdown window.

Use Case 2: Inventory Onboarding and SKU Lookup

- Trigger: Product registration, barcode reading, or inventory restock.
- Process Flow:
 - a. The warehouse operator inputs or scans an alphanumeric SKU/Barcode.
 - b. The system runs indexed reads against local inventory data.
 - c. If a new item is being added, a standard record insertion maps the Arabic name, cost metrics, and low-stock warning criteria.
 - d. All financial fields (prices, costs) are strictly validated to prevent the usage of floating-point numbers.

Use Case 3: The Atomic POS Checkout Transaction

- Trigger: Totaling a shopping cart and printing a merchant receipt.

- Process Flow:
 - a. The cashier clicks "Complete Checkout". The system immediately initiates an isolated database transaction block.
 - b. Inventory Threshold Check: The system verifies that the current local inventory count for every item in the checkout cart is equal to or greater than the requested purchasing amount. If any single product falls short, the transaction is instantly canceled, rolling back any initial steps and displaying a clear warning.
 - c. Invoice Generation: A primary invoice ledger record is created, tracking numbers, customer associations via `client_id`, dates, and finalized totals.
 - d. Line Item Breakdown: Individual items, exact purchasing quantities, item discounts, and applied tax percentages are mapped directly back to the parent invoice database entry.
 - e. Inventory Stock Deduction: The local inventory count for each purchased product is automatically decremented inside the transaction block.
 - f. Ledger Reconciliation: If the invoice is fully settled (`PAID`), an income transaction entry is appended to the operational cash log. If the invoice is deferred or partially funded (`PARTIAL` / `UNPAID`), a liability entry is opened in the customer accounts receivable ledger under `debt_ledger` mapping to that client.
 - g. The database transaction concludes. If any error occurs during steps 1 through 6, the local engine aborts the routine, preventing broken, half-written data entries.

Use Case 4: Debt Management & Settlement

- Trigger: Customer returns to repay an open account balance.
- Process Flow:
 - a. The operator queries the client database or opens the customer's profile via a phone or name lookup within the app to review their current outstanding obligations.
 - b. The merchant registers a partial or complete cash payment against their debt profile.
 - c. The system decreases the customer's total liability score within the `debt_ledger` records linked via `contact_id`.
 - d. The related parent invoice transitions its operational status indicator (e.g., from `UNPAID` to `PAID`).
 - e. A new transaction entry is injected into the cash ledger tracking pipelines, logging a positive cash inflow.

4. Production-Ready Local Database Schema

Developers must configure the local SQLite database file using the layout below. This schema enforces strict data types, relational references, and operational indexing.

SQL

```
-- ===== BARKAHFLOW STANDALONE OFFLINE-FIRST RELATIONAL
SCHEMA-- ===== Execution Directives: Run immediately
upon opening the native database file.
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;
```

```

-- ----- TABLE: products-- Description: Core inventory
records, tracking prices and alert triggers.-- -----
CREATE TABLE IF NOT EXISTS products (
  id          TEXT PRIMARY KEY,          -- Secure RFC4122 UUID
  sku         TEXT UNIQUE NOT NULL,      -- Alphanumeric Barcode/SKU string
  name        TEXT NOT NULL,             -- Primary Display Title (supports Arabic/English)
  cost_price  INTEGER NOT NULL,          -- Stored in lowest currency denomination (e.g., Fils, Cents, Halalas)
  retail_price INTEGER NOT NULL,          -- Stored in lowest currency denomination (e.g., Fils, Cents, Halalas)
  stock_qty   INTEGER NOT NULL DEFAULT 0, -- Available inventory units
  alert_threshold INTEGER NOT NULL DEFAULT 5, -- Low-stock trigger minimum level
  tax_rate    REAL DEFAULT 0,            -- Decimal percentage multiplier (e.g., 0.15 for 15% VAT)
  updated_at  TEXT NOT NULL              -- ISO8601 Timestamp String
);

-- ----- TABLE: clients-- Description: Customer profile
information for POS reference, attribution, and debt tracking.-- -----
-----CREATE TABLE IF NOT EXISTS clients (
  id          TEXT PRIMARY KEY,          -- Unique Client UUID
  name        TEXT NOT NULL,            -- Full Name of the client
  phone       TEXT UNIQUE,              -- Phone number for rapid terminal searching
  email       TEXT,                    -- Optional email address
  notes       TEXT,                    -- Miscellaneous merchant contextual notes
  created_at  TEXT NOT NULL,            -- ISO8601 Timestamp String
  updated_at  TEXT NOT NULL             -- ISO8601 Timestamp String
);

-- ----- TABLE: invoices-- Description: Financial
parent records representing distinct client sales.-- -----
--CREATE TABLE IF NOT EXISTS invoices (
  id          TEXT PRIMARY KEY,          -- Unique Invoice UUID
  invoice_number TEXT UNIQUE NOT NULL,    -- Human-readable sequential reference code
  client_id   TEXT REFERENCES clients(id) ON DELETE SET NULL, -- Link to internal customer profile identifier
  subtotal    INTEGER NOT NULL,          -- Total before tax and discounts (in absolute integer units)
  tax         INTEGER NOT NULL DEFAULT 0, -- Computed tax value (in absolute integer units)
  discount    INTEGER NOT NULL DEFAULT 0, -- Applied reduction value (in absolute integer units)
  total       INTEGER NOT NULL,          -- Finalized total payable cash value
  status      TEXT NOT NULL CHECK(status IN ('PAID', 'PARTIAL', 'UNPAID')),
  created_at  TEXT NOT NULL,            -- ISO8601 Timestamp String
  updated_at  TEXT NOT NULL             -- ISO8601 Timestamp String
);

-- ----- TABLE: line_items-- Description: Relational
join mapping individual items to invoices.-- -----CREATE
TABLE IF NOT EXISTS line_items (
  id          TEXT PRIMARY KEY,          -- Line Item UUID
  invoice_id  TEXT NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
  product_id  TEXT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  qty         INTEGER NOT NULL,          -- Volume purchased
  unit_price  INTEGER NOT NULL,          -- Price of item at the point of sale
  discount    INTEGER DEFAULT 0,         -- Specific markdown applied to line
  subtotal    INTEGER NOT NULL           -- Evaluated final sum: (qty * unit_price) - discount
);

-- ----- TABLE: transactions-- Description: Core
financial ledger tracking cash inflows and outflows.-- -----
--CREATE TABLE IF NOT EXISTS transactions (

```

```

    id            TEXT PRIMARY KEY,          -- Ledger entry UUID
    type          TEXT NOT NULL CHECK(type IN ('INCOME', 'EXPENSE')),
    amount        INTEGER NOT NULL,          -- Transaction absolute monetary value
    source_type   TEXT CHECK(source_type IN ('invoice', 'manual')), -- Origin tracking identifier
    source_id     TEXT,                      -- Contains target invoice_id if linked to a sale
    category      TEXT,                      -- Operational tag (e.g., 'Inventory Supply', 'Rent', 'Sales Inflow')
    transaction_date TEXT NOT NULL,          -- ISO8601 Date String
    notes         TEXT,                      -- Freeform merchant comments
    created_at    TEXT NOT NULL              -- ISO8601 Timestamp String
);

-- ----- TABLE: debt_ledger-- Description: Customer
account logs tracking outstanding credit and liabilities.-- -----
-----CREATE TABLE IF NOT EXISTS debt_ledger (
    id            TEXT PRIMARY KEY,          -- Debt Ledger UUID
    type          TEXT NOT NULL CHECK(type IN ('RECEIVABLE', 'PAYABLE')),
    contact_id    TEXT NOT NULL REFERENCES clients(id) ON DELETE RESTRICT, -- Maps to client table for receivables
    total_debt    INTEGER NOT NULL,          -- Initial outstanding balance amount
    remaining_debt INTEGER NOT NULL,          -- Current outstanding balance amount
    status        TEXT NOT NULL CHECK(status IN ('ACTIVE', 'SETTLED', 'PARTIAL')),
    invoice_id    TEXT REFERENCES invoices(id) ON DELETE SET NULL, -- Linked transaction anchor
    created_at    TEXT NOT NULL,             -- ISO8601 Timestamp String
    updated_at    TEXT NOT NULL              -- ISO8601 Timestamp String
);

-- ===== PERFORMANCE OPTIMIZATION INDEXES--
=====-- Speeds up barcode scanning and immediate point
checkout operations
CREATE INDEX IF NOT EXISTS idx_products_sku ON products(sku);

-- Accelerates partial text matching search inputs on checkout terminals
CREATE INDEX IF NOT EXISTS idx_products_name ON products(name);

-- Accelerates customer lookup via phone strings at the register
CREATE INDEX IF NOT EXISTS idx_clients_phone ON clients(phone);

-- Speeds up search queries for client names on the terminal or CRM dashboard
CREATE INDEX IF NOT EXISTS idx_clients_name ON clients(name);

-- Speeds up financial reporting aggregations and administrative reviews
CREATE INDEX IF NOT EXISTS idx_invoices_status ON invoices(status, created_at);

```

5. Architectural Safeguards and Standards

To ensure the long-term integrity of the system, all developers and engineering interns must adhere to the following strict technical boundaries during implementation:

5.1 Financial Precision Mandate

- The Anti-Float Rule: Financial values must never use floating-point numbers (`FLOAT` , `REAL` , `DOUBLE`). Due to the way computers handle binary fractions (IEEE 754 standards), using floats creates rounding errors over time.
- The Integer Enforcer: All currency calculations must be scaled and handled strictly as whole integers representing the smallest native subunit of local currency. For example, a transaction totaling 12.50 MAD

(Moroccan Dirham) must be processed, calculated, and stored in the database as 1250 (Centimes/Fils/Halalas). Conversion to a decimal format is handled exclusively in the UI rendering layer.

5.2 Transaction Isolation Integrity

- The Transaction Guard: Every inventory change, customer billing event, client profile update, and ledger mutation must execute inside an explicit `BEGIN IMMEDIATE TRANSACTION` container block.
- Atomic Rollbacks: If an operator attempts to purchase three units of an item while the local database registers only two, the engine must immediately abort the pipeline and invoke a full `ROLLBACK`. This protects the relational data structure from half-written or orphaned invoice components.

5.3 Write-Ahead Logging (WAL) Mode

- Performance Profile: The initialization routine must immediately activate SQLite's Write-Ahead Logging mode (`PRAGMA journal_mode = WAL;`).
- Concurrency Architecture: By switching to WAL mode, database reads and writes no longer block one another. This allows the terminal UI to run reporting queries or low-stock checks in the background while the frontend completes new checkouts without experiencing UI delay or database lockup.